Practical Assignment 3

Problem Statement :

Consider the telephone book database of N clients. Make use of a hash table implementation to quickly look up a client's telephone number. Make use of linear probing, double hashing and quadratic collision handling techniques.

Input code:

```
#include<stdio.h>
#include<string.h>
#define size 10
int i;
typedef struct
{
    char name[30];
    long phone;
    int occupied;
}client;
client hashtable[size];
void insertlinear(char *name, long phone);
void insertquadratic(char *name, long phone);
void insertdouble(char*name, long phone);
void search(char *name);
void display();
int hashfunction(char name[]);
int hashfunction(char name[])
 {
    int hash=0;
    for(int i=0;name[i];i++)
    {
```

```
        hash=(hash+name[i]%size);
    }
    return hash;
}
int secondhash(char*name)
{
    int hash=0;
    for(int i=0;name[i];i++)
    {
        hash+=name[i];
        return 7-(hash%7);

    }
}
void main()
{
    int ch;
    char name[30];
    long phone;
    do
    {
        printf("\nMENU\n1.Linear probing\n2.Quadratic probing\n3.Double
hashing\n4.Searching\n5.Display\n6.Exit");
        printf("\nEnter your choice:");
        scanf("%d",&ch);
        switch (ch)
        {
        case 1:
           printf("Enter name and phone number:");
```

```
            scanf("%s%ld",&name,&phone);
            insertlinear(name,phone);
            break;
        case 2:
             printf("enter name and phone");
             scanf("%s%ld",&name,&phone);
             insertquadratic(name,phone);
            break;
        case 3:
             printf("Enter name and phone number:");
            scanf("%s%ld",&name,&phone);
            insertdouble(name,phone);
            break;
        case 4:
            printf("Enter name to search:");
            scanf("%s",&name);
            search(name);
            break;

        case 5:
             display();
             break;

        default:
            break;
            }
    }while(ch!=5);
}
```

```
void insertlinear(char *name,long phone)
{
    int index=hashfunction(name);
    int start=index;
    while(hashtable[index].occupied)
    {
        index=(index+1)%size;
        if(index==start)
          {
              printf("Hash table is full");
               return;
          }
    }
    strcpy(hashtable[index].name,name);
    hashtable[index].phone=phone;
    hashtable[index].occupied = 1;
    printf("Inserted using linear probing\n");

}
void insertquadratic(char *name, long phone)
{
    int index=hashfunction(name);
    while(hashtable[(index + i*i)%size].occupied&& i<size)
    {
        i++;
    }
    if(i==size)
    {
        printf("Hash table is full");
```

```
        return;
    }
    index=(index + i*i)%size;
    strcpy(hashtable[index].name,name);
    hashtable[index].phone=phone;
    hashtable[index].occupied=1;
    printf("Inserted using quadratic probing\n");
}
void insertdouble(char*name,long phone)
{
    int index=hashfunction(name);
    int step=secondhash(name);
    int start=index;
    while(hashtable[index].occupied)
    {
        index=(index+step)%size;
        if (index==start)
        {

            printf("table full");
            return;
        }
    }
    index=(index + i*i)%size;
    strcpy(hashtable[index].name,name);
    hashtable[index].phone=phone;
    hashtable[index].occupied=1;
    printf("Inserted using double hashing\n");
}
```

```
void search(char *name)
{
    int index=hashfunction(name);
    int start=index;
    while(hashtable[index].occupied)
    {
          if(strcmp(hashtable[index].name,name)==0)
        {
            printf("phone number of %s is %ld",name,hashtable[index].phone);
            return;
        }
        index=(index+1)%size;
        if(index==start)
            {
            break;
            }
    }
            printf("clinet not found");



}
void display()
{
    int i;
    printf("Hash Table");
    printf("%-5s %-30s %-15s\n", "Index", "Name", "Phone");
    for(i = 0; i < size; i++)
    {
        if(hashtable[i].occupied)
```

```
        {
            printf("%-5d %-30s %-15ld\n", i, hashtable[i].name, hashtable[i].phone);
        }
        else
        {
            printf("%-5d %-30s %-15s\n", i, "---", "---");
        }
    }
}
```

Output:

```
--- HASH TABLE MENU ---
1. Linear Probing
2. Quadratic Probing
3. Double Hashing
4. Searching
5. Display
6. Exit
Enter your choice: 1
Enter name and phone number: Ankush 9847562103
Inserted 'Ankush' at index 8 using Linear Probing.


--- HASH TABLE MENU ---
1. Linear Probing
2. Quadratic Probing
3. Double Hashing
4. Searching
```

5. Display

6. Exit

Enter your choice: 2

Enter name and phone number: Aradhya 8975641235

Inserted 'Aradhya' at index 9 using Quadratic Probing.

--- HASH TABLE MENU ---

1. Linear Probing

2. Quadratic Probing

3. Double Hashing

4. Searching

5. Display

6. Exit

Enter your choice: 3

Enter name and phone number: Aditi 7985664125

Inserted 'Aditi' at index 1 using Double Hashing (Step = 6).

--- HASH TABLE MENU ---

1. Linear Probing

2. Quadratic Probing

3. Double Hashing

4. Searching

5. Display

6. Exit

Enter your choice: 4

Enter name to search: Aradhya

Found! Phone number of Aradhya is 385706643 (Index: 9)

--- HASH TABLE MENU ---

1. Linear Probing

2. Quadratic Probing

3. Double Hashing

4. Searching

5. Display

6. Exit

Enter your choice: 5

Index	Name	Phone
0	---	---
1	Aditi	-604270467
2	---	---
3	---	---
4	---	---
5	---	---
6	---	---
7	---	---
8	Ankush	1257627511
9	Aradhya	385706643

--- HASH TABLE MENU ---

1. Linear Probing

2. Quadratic Probing

3. Double Hashing

4. Searching

5. Display

6. Exit

Enter your choice: 6

Exiting program...

Process returned 0 (0x0)   execution time : 1451.468 s

Press any key to continue.